How I Use AI as an Economist

Guohui Jiang

Last updated: April 12, 2026 · gjiang-economics.github.io

I believe AI literacy is becoming a fundamental skill in an AI-driven economy. This document describes how I use AI tools in my day-to-day work as an academic economist. I focus on practical tips that I have found genuinely useful. It is updated periodically as I learn and experiment with new things.

Contents

1 General Tips (Any AI Tool) **2**

2 Claude Code **2**

2.1 Getting Started 3

2.2 Claude Code for Research 3

2.3 Claude Code for Administrative Work 7

3 Chat Bot **8**

3.1 Learning in an AI-Driven Economy 8

3.2 Brainstorming Research 8

4 Resources **8**

General Tips (Any AI Tool)

▶ Dictation with Wispr Flow

Speaking is much faster than typing. I use Wispr Flow, which transcribes speech in real time. It dramatically speeds up my interaction with AI—I can dictate a paragraph-long prompt in seconds instead of typing it out.

Handling accent and transcription errors. If you are not a native English speaker, speech-to-text will occasionally mishear words or drop articles. In Claude Code, I solve this with a *hook*—a script that runs every time you submit a prompt, telling Claude the input was dictated so it silently corrects artifacts instead of asking for clarification.

Save this as `~/.claude/hooks/dictation-context.sh`:

```
#!/bin/bash
cat <<'EOF'
DICTATION NOTICE: The user's prompt was likely dictated
using Wispr Flow (speech-to-text). The user is a native
Chinese (Mandarin) speaker. The transcription may contain:
- Homophones or near-homophones (e.g. "EA" -> "it")
- Misheard technical terms or proper nouns
- Missing or wrong articles, prepositions, or plurals
- Run-on sentences or missing punctuation

Mentally correct any likely transcription artifacts and
interpret the intended meaning. Do not ask for clarification
on obvious transcription errors -- just fix them silently.
EOF
```

Then activate it in `~/.claude/settings.json`:

```
{
  "hooks": {
    "UserPromptSubmit": [
      {
        "type": "command",
        "command": "$HOME/.claude/hooks/dictation-context.sh"
      }
    ]
  }
}
```

Adapt the script to your own native language—just change “Chinese (Mandarin)” and the example error patterns to match your accent.

Claude Code

Claude Code is a terminal-based AI assistant that can read your files, write code, run commands, and interact with your project directly. Watch this video by Paul Goldsmith-Pinkham for a quick

introduction, or see the official documentation for installation and a full overview.

Getting Started

▶ My Setup: VS Code + Terminal CLI

I use Claude Code through the **terminal CLI inside VS Code**:

- **VS Code** lets me see the folder structure in the sidebar, open files, and track what Claude modifies.
- **The terminal CLI** (run `claude` in VS Code's integrated terminal) is where I interact with Claude Code. The CLI uses significantly fewer tokens than the VS Code extension or desktop app.

To set this up: open your project folder in VS Code, then open the integrated terminal (`Ctrl+``) and type `claude`.

▶ First Thing to Do: Run `/init`

Run `/init` in any new project. This generates a `CLAUDE.md` file—a plain-text briefing sheet describing your project, preferences, and instructions the AI should follow every session. Without it, you re-explain context every time.
Example:

```
# Project: My Research Paper

## What this project is
Analysis of WWII draft records using Stata and R.

## My preferences
- Use Stata for data cleaning, R for figures
- Commit changes with descriptive messages
- Ask before deleting files
- I use dictation, so prompts may be conversational
```

Claude Code for Research

▶ Treat Claude Code as an RA for Your Entire Life

Claude Code combines high execution power (reads files, writes code, runs commands) with high intelligence (reasons, plans, adapts). This makes it a research assistant for *anything*—not just coding, but admin, writing, data work, travel logistics.
The key insight: even if the first result is mediocre, it is worth teaching Claude Code how to do the task well. Every skill you build, every memory you save, every line in your `CLAUDE.md` makes every future session better—across every project, on every machine, forever. Not sure how to teach it? Just ask:

```
I want Claude Code to do [X] better in the future.
What's the best way to achieve this -- a skill,
a hook, a memory, a CLAUDE.md entry, or something else?
```

▶ Plan Mode for Complex Tasks

For multi-step tasks, start in *plan mode* (`Shift+Tab`). Claude Code proposes a structured plan before making changes. You review and approve it first, then it executes. This prevents the AI from charging ahead in the wrong direction.

```
I want to restructure this Stata project so that
each .do file handles one step of the pipeline
(clean, merge, analyze, figures). Plan this out first.
```

▶ The Reflection Workflow

Perhaps the most powerful technique I have found. When you encounter a new type of task:

1. **Push Claude Code to its best.** Work through the task iteratively.
2. **Do not delete the session history.** The full history is essential for what comes next.
3. **Ask Claude Code to reflect:**

```
Reflect on this entire session. Is there anything you
should save as a skill, a memory, or a setting change
so that next time we do this, it goes faster?
```

Claude Code will then create reusable artifacts—skills, memories, or configuration changes. The first time you do a task with AI might not save time. The second time, it will. For more on creating skills, see this walkthrough.
I have built a dedicated reflection skill that automates this process. Install it with:

```
mkdir -p ~/.claude/skills/reflection && \
curl -o ~/.claude/skills/reflection/SKILL.md \
  https://gjiang-economics.github.io/skills/reflection.md
```

Then at the end of any session, type `/reflection`.

▶ Structured Workflow with Compound Engineering

For non-trivial tasks, I use the Compound Engineering plugin, which structures work into four steps:

1. **Plan** (`/ce:plan`)—Scope, decisions, file targets, test scenarios, and risks—before

touching code.

2. **Work** (`/ce:work`)—Execute the plan systematically.
3. **Review** (`/ce:review`)—Dispatch independent reviewers that start with a blank slate, catching issues the working agent would rationalize away.
4. **Compound** (`/ce:compound`)—Document the solution while context is fresh.

The review step is particularly valuable. The agent that built something is the least likely to notice what it missed—like an author proofreading their own paper.

► Cross-Model Brainstorming

When Claude Code struggles with a subtask after a few attempts, try a "second opinion" from another model:

1. Ask Claude Code to **summarize the situation**: problem, what was tried, what failed.
2. Say: *"Give me a self-contained prompt I can paste into Gemini."*
3. Copy it into Gemini (or another AI), then paste the solution back.

Different models have different strengths. Combining them often gets you to a solution faster than pushing a single model repeatedly.

► Remote Control

Claude usage is calculated in **5-hour windows**. If Claude Code pauses for a permission prompt while you are away, those hours are wasted. *Remote Control* lets you monitor and approve sessions from your phone or any browser via claude.ai/code or the Claude mobile app (iOS, Android).

```
/remote-control
```

Or start a new session with: `claude -remote-control`
My workflow. Before leaving the office, I start a long task with `/remote-control`. On the train home, I check my phone, approve prompts, and review progress. If I know what comes next, I start a second session—when session one completes, I pick up session two from my phone. See the official documentation for details.

► Syncing Claude Code Across Machines

I use Claude Code on a MacBook and a Windows desktop. To keep them identical, I store shared config in Dropbox and symlink from `~/.claude/`:

```
Dropbox/AI_meta/claude-sync/
|-- memory/        # Your personal AI memory
|-- skills/        # Custom skills
|-- commands/      # Custom slash commands
|-- plugins/       # Plugins
|-- CLAUDE.md      # Global instructions
|-- settings.json  # Shared preferences
|-- .mcp.json      # MCP server config
```

On Mac:

```
ln -sf ~/Dropbox/AI_meta/claude-sync/memory   ~/.claude/memory
ln -sf ~/Dropbox/AI_meta/claude-sync/skills   ~/.claude/skills
# ... same for each item
```

On Windows (PowerShell as Administrator):

```
# Folders -- use Junction
New-Item -ItemType Junction -Path "$env:USERPROFILE\.claude\memory" `
  -Target "C:\Users\YOU\Dropbox\AI_meta\claude-sync\memory"

# Files -- use SymbolicLink
New-Item -ItemType SymbolicLink -Path "$env:USERPROFILE\.claude\CLAUDE.
    md" `
  -Target "C:\Users\YOU\Dropbox\AI_meta\claude-sync\CLAUDE.md"
```

Machine-specific settings go in `settings.local.json` and do not sync.

► Remove AI-Generated Writing Patterns

AI text has telltale patterns—inflated language, filler phrases, em dashes everywhere. The humanizer skill detects 29 such patterns and rewrites text to sound natural.
Claude Code. Install and run `/humanizer`:

```
mkdir -p ~/.claude/skills/humanizer && \
git clone https://github.com/blader/humanizer.git \
  ~/.claude/skills/humanizer
```

Claude AI (web) & Claude Desktop. Create a Project, then paste the SKILL.md contents into the Project's custom instructions.

► Hooks: Automatic Guardrails and Context Injection

Hooks are shell scripts that Claude Code runs automatically on specific events—prompt submission, tool execution, file edits. You configure them once and they enforce rules in every session.

- **Inject context**—The dictation hook above tells Claude your prompt was speech-to-text.
- **Block dangerous actions**—Intercept tool calls to prevent force-pushes or unapproved deletions.
- **Automate repetitive steps**—Run linting or update changelogs automatically.

For a curated collection, see this tweet by @zodchiii. Full documentation: official hooks docs.

Claude Code for Administrative Work

▶ **Travel Reimbursement**

After a conference, I use Claude Code to prepare reimbursement documents using a custom *skill*.

Step 1: Organize your folder.

```
reimbursement/
|-- proofs/                        # Receipts and payment evidence
|   |-- 01 flight receipt.pdf
|   |-- 01 boarding pass.pdf
|   |-- 03 hotel invoice.pdf
|   |-- 06 taxi receipt.pdf
|   |-- 11 city tax photo.JPG      # Photos of paper receipts work too
|-- host_reimbursement_policy.pdf   # Instructions from the host
|-- summary_template.xlsx           # Excel template (if any)
```

Files in `proofs/` are numbered by item. Files sharing the same prefix (e.g., `01`) belong to the same expense.

Step 2: Install the skill.

```
mkdir -p ~/.claude/skills/reimburse-external && \
curl -o ~/.claude/skills/reimburse-external/SKILL.md \
  https://gjiang-economics.github.io/skills/reimburse-external.md
```

The skill walks Claude through a 10-phase workflow: read the host's policy, catalog receipts, verify against rules, calculate the claim with currency conversion, fill in templates, merge receipts into a single PDF, and draft the submission email. View the full skill file.

Step 3: Run it. Open the reimbursement folder in VS Code and type:

```
/reimburse-external
```

Adapting this. This skill handles *external* reimbursement (conference hosts, funding bodies). For your university's process, create a separate skill—ask Claude Code to reflect after your first reimbursement and it will build one tailored to your institution.

Chat Bot

Learning in an AI-Driven Economy

▶ Use AI to Design a Self-Guided Study Plan

Pick a topic you want to learn. Tell the AI your background, available time, and what you want to understand. Ask it to build a study plan—readings, key ideas, and an order to work through them. As you read, use the AI to ask questions, check your understanding, and explore connections. What used to take days of searching now takes hours.
This idea comes from a post by Jesús Fernández-Villaverde, who used AI to design a self-guided study of sociologist Erving Goffman—and found the experience comparable to a master's-level course week.

Brainstorming Research

▶ Avoid Sycophancy

AI chatbots trained with RLHF tend to be *sycophantic*—they default to agreement rather than honest critique. This is counterproductive when brainstorming research ideas.
The fix (Claude). Go to **Settings** → **General** → **"What personal preferences should Claude consider in responses?"** and add:

```
Be direct and honest. Don't default to agreement
or flattery: I prefer candid feedback, especially
on my research.
```

This overrides the default tendency at the preference level, so every conversation starts candid rather than agreeable.

Resources

- In Claude Code CLI, type `/powerup`
- Claude Blattman
- Paul Goldsmith-Pinkham at Markus Academy
- Aniket Panjwani at VoxDev, and his YouTube channel
- awesome-ai-for-economists